

Mock - Teams

- A. 1) HT, m pos, open addressing, n elem.
 Search for $e \rightarrow$ is it possible that during the search more than n positions are checked?

If $m > n$, yes.

In open addressing $m \geq n$. (each elem. occupies 1 slot)

If e is found $\Rightarrow$ search stops.

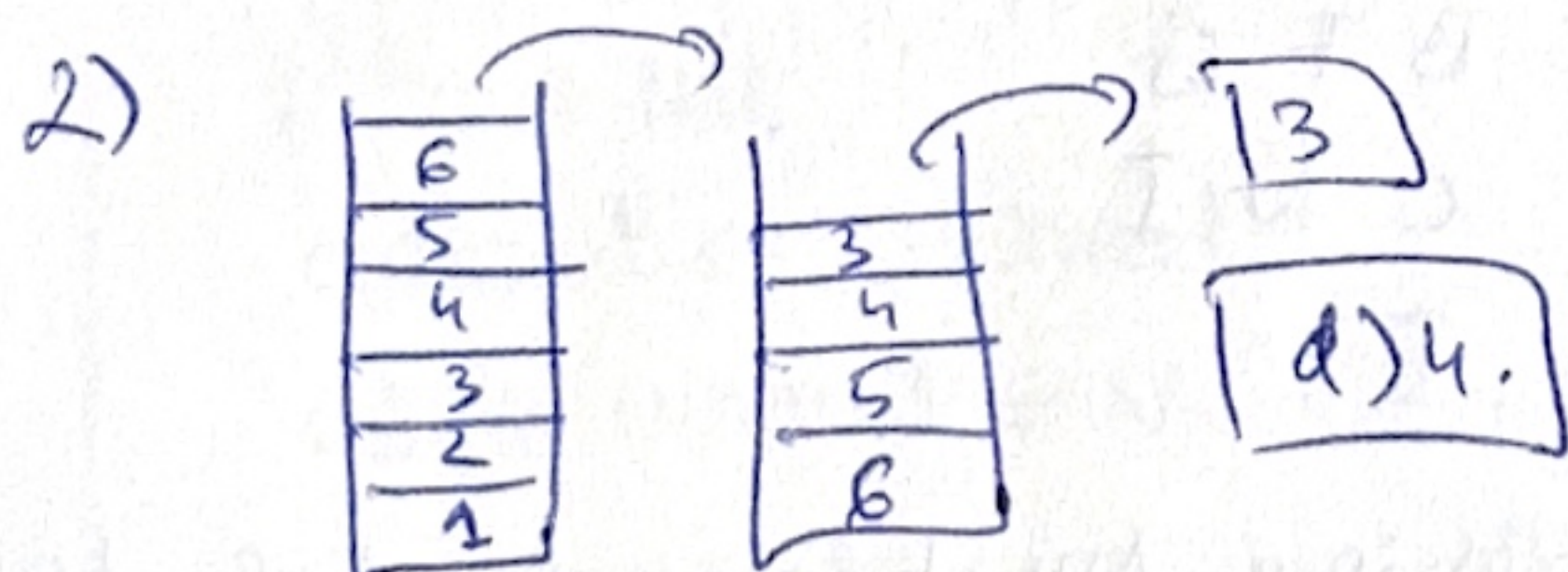
If e is not found $\Rightarrow$ search continues (exceeds n and goes to m)

20
30
40
.
.
.

search 45 $\Rightarrow$?

$n = 3, m = 6$

b) only if e not in HT.



- 3) 4 6 7 + 1 - 2 5 * + +

Token	Stack
4	4
6	4, 6
7	4, 6, 7 } $\Rightarrow 6 + 7 = 13$
+	4, 13
1	4, 13, 1 } $\Rightarrow 13 - 1 = 12$
-	4, 12
2	4, 12, 2
5	4, 12, 2, 5 } $\Rightarrow 2 * 5 = 10$
*	4, 12, 10 } $\Rightarrow 12 + 10 = 22$
+	4, 22
+	<u>26</u>

- 4) a) $O(n^4)$
 d) $\Omega(n^3)$
 e) $\Omega(1)$

B. 1) BT → preorder, postorder, inorder, level order.

• preorder: ~~LR~~ ~~R~~ ~~O~~ Root-Left-Right



D A B C E H F G N I. ✓

• postorder: Left-Right-Root



E C B A ~~H~~ ~~F~~ ~~G~~ F H I G H D ✓

• inorder: Left-Root-Right



B E C A D F H N G I ✓

• levelorder:

D A H B F G C N I E ~

D
A H
B F G
C N I
E

2) HT, $m=8$ pos, div. method., sep. chaining, AVL tree; load factor = $\frac{n}{m} = \frac{9}{8} = \alpha$

$$\frac{n}{m} = \frac{9}{8}$$

$$h(8) = 8 \% 8 = 0$$

$$h(99) = 99 \% 8 = 3$$

$$h(40) = 40 \% 8 = 0$$

$$h(19) = 3$$

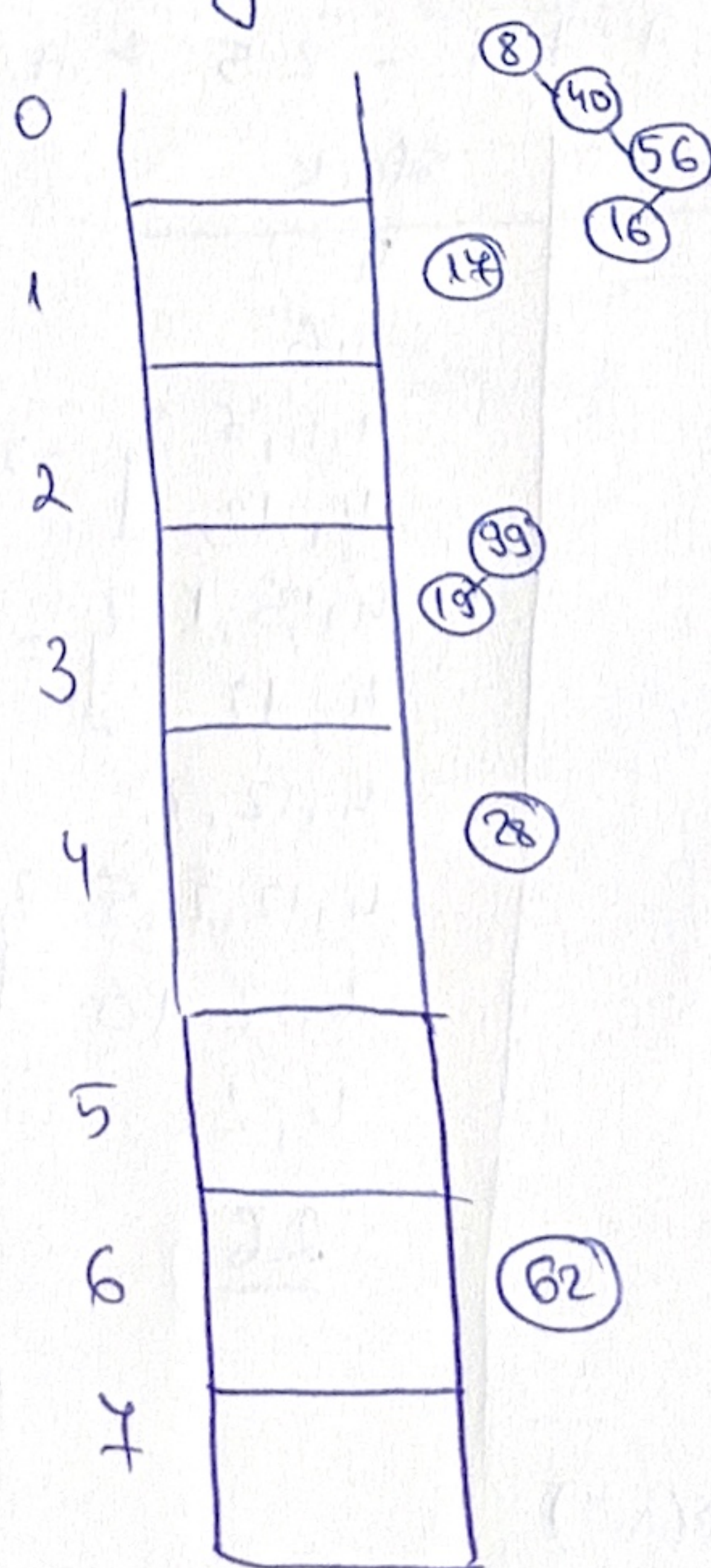
$$h(56) = 0$$

$$h(16) = 0$$

$$h(17) = 1$$

$$h(28) = 4$$

$$h(62) = 6$$



3) see mock exam ①

Reverse SL - in linear time

- $\Theta(1)$ E.S. complexity
- repres. of SL
- impl.
- complexity justif.

Representation:

Node:

info: TElem

next: $\uparrow$ Node

SL:

head: $\uparrow$ Node

Implem:

subalgorithm reverseSL(sll) is

prev $\leftarrow$ Nil

current $\leftarrow$ sll.head

while current $\neq$ Nil execute

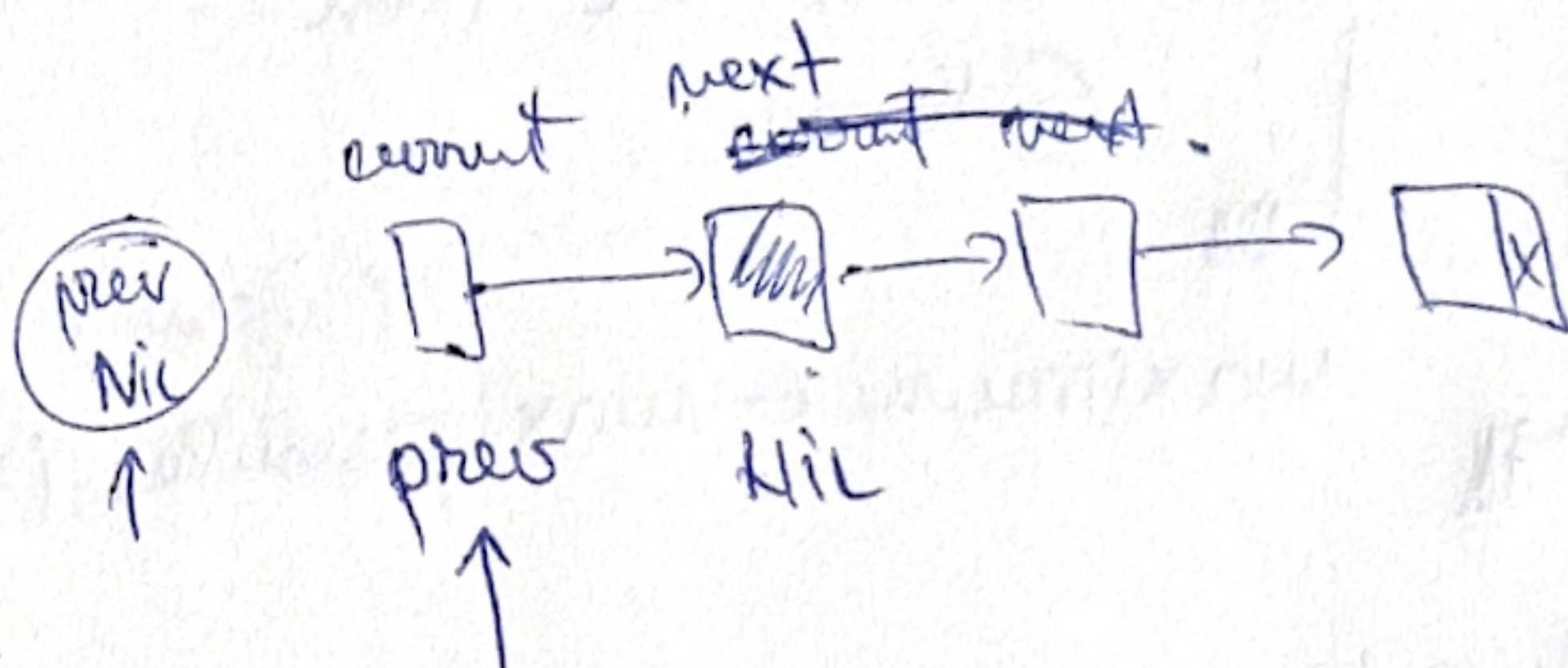
 nextNode $\leftarrow$ [current].next

 [current].next $\leftarrow$ prev

 prev $\leftarrow$ current

 current $\leftarrow$ nextNode

sll.head $\leftarrow$ prev // last visited node



For each node, save its old next, reverse the link, advance to next node.

$\Theta(n) = BC = WC$ because the while loop visits each node once.

ES: $\Theta(1)$ because we only use 3 pointers: prev, nextNode, current.

2) Max. value in a binary tree containing int. nr.

Give repres. of binary tree (don't memorize parent of a node)

Implem.

Complexity

Aux containers $\rightarrow$ specify

BTNode:

info: Integer

left: $\uparrow$ BTNode

right: $\uparrow$ BTNode

BT:

root: $\uparrow$ BTNode

function maximum (tree) is

if tree.root = Nil then

@throw...

maximum $\leftarrow$ maximumRec (tree.root)

function maximumRec (node) is

maxValue $\leftarrow$ [node].info

if [node].left $\neq$ Nil then

leftMax $\leftarrow$ maximumRec ([node].left)

if leftMax > maxValue then

maxValue $\leftarrow$ leftMax

if [node].right $\neq$ Nil then

rightMax $\leftarrow$ maximumRec ([node].right)

if rightMax > maxValue then

maxValue $\leftarrow$ rightMax

maximumRec $\leftarrow$ maxValue

Bc = $\Theta(n)$

Wc = $\Theta(n)$

Total: $\Theta(n)$

Not a BST tree, so we need to visit each branch.

ES: $\Theta(h)$ from recursion stack; No aux containers.

ADT Quartiler

- $\text{init}(g) - \Theta(1)$
- $\text{add}(g, e) - O(\log_2 m)$ amortized
- $\text{getTopQuartile}(g) - \Theta(1)$ total
- $\text{deleteTopQuartile}(g) - O(\log_2 m)$ total.

PERCENTILE: $\rightarrow$ sort elems

$\rightarrow$ the 75th percentile: elem around pos. $[0.75 \cdot m]$

Choose DS:

$\text{add} - O(\log_2 m)$

$\text{get} - \Theta(1)$

$\text{deleteTop} \dots - O(\log_2 m)$

$\rightarrow$ HEAP

min-heap for top 25% (high)

max-heap for bottom 75% (low)

Representation:

Quartiler:

low: MaxHeap

high: MinHeap

size: Integer

MinHeap:

elems: Integer[]

size: Integer

Invariant:

$\text{size}(\text{low}) = \text{ceil}(3 \cdot m / 4)$

$\text{size}(\text{high}) = m - \text{size}(\text{low})$

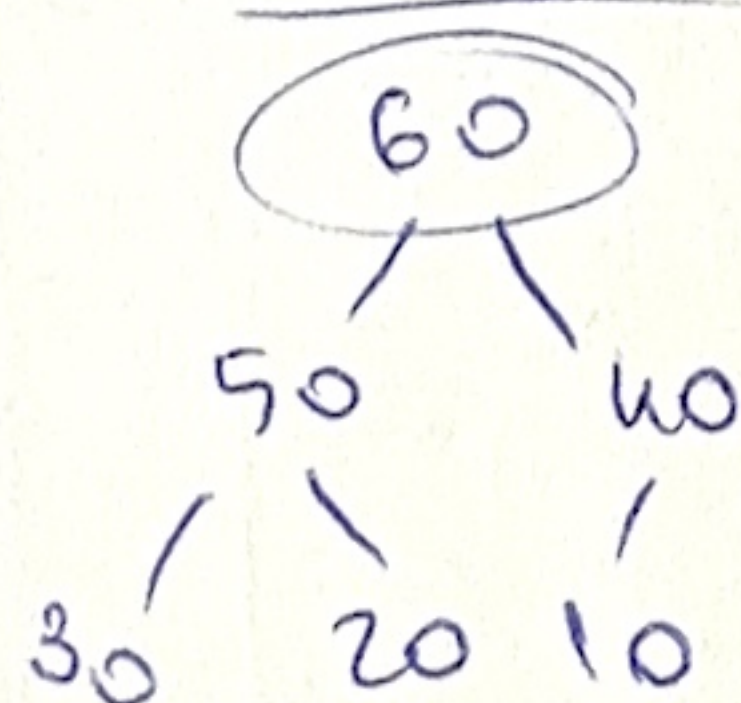
MaxHeap

elems: Integer[]

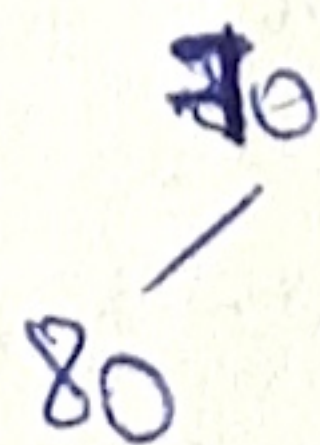
size: Integer

2. 10, 20, 30, 40, 50, 60 | 70, 80.

$\rightarrow$ low:



high:



$\Rightarrow \text{topQuartile} = 60$

3. $\text{init}(g) \rightarrow$ create both heaps, $\Theta(1)$

(MAX)

$\text{add}(g, e) \rightarrow$ If low is empty or $\text{elem} \leq \text{root}(\text{low})$, add to low, otherwise add to high.

$\rightarrow$ Rebalance - (move to high) if low has too many elems.

- move to low if high low has too few elems.

$\rightarrow$ heap add/remove = $O(\log_2 m)$ amortized.

$\text{getTopQuartile} \rightarrow \text{root}(\text{low})$ return $\Rightarrow \Theta(1)$, no traversal

$\text{deleteTopQuartile} \rightarrow$ remove root from low heap $\Rightarrow$ rebalance $\Rightarrow$

$\Rightarrow$ move root between heaps $\Rightarrow$ remove + add $\Rightarrow O(\log_2 m)$
 $\downarrow$
 $O(\log_2 m)$ $O(\log_2 m)$

Subalgorithm deleteTopQuartile(2) is

if $g.m = 0$ then $\rightarrow O(\log_2 m)$

throw exception

removeMax(g.low)

$g.m \leftarrow g.m - 1$

requiredLow $\leftarrow \text{ceil}(3 * g.m / 4)$

while $g.\text{low.size} < \text{requiredLow}$ ex

$x \leftarrow \text{removeMin}(g.\text{high})$

addMax(g.low, x)

removeMax(maxHeap)

swap root with last elem

$\text{size} \leftarrow \text{size} - 1$

heapifyDown(root)

removeMin(minHeap)

$u \leftarrow u$

$\rightarrow O(\log_2 m)$

$\rightarrow O(\log_2 m)$

because a
 Binary Heap
 has height
 $O(\log_2 m)$ and
 heapifyDown(
 traverses at most
 one root-to-leaf
 path